

MEF ფაილის ფორმატი

1. MEF — Minimal Executable Format

MEF (Minimal Executable Format) — ეს არის მარტივი, სასწავლო მიზნებისთვის შექმნილი გამშვები ფაილის ფორმატი. ისევე, როგორც Linux-ზე ELF ან Windows-ზე PE, MEF-ი ასწავლის, თუ როგორ "გასაგები" ხდება კომპიუტერისთვის ბინარული ფაილი.

1.1 რატომ გვჭირდება ფაილის ფორმატი?

როდესაც პროგრამა გაეშვება, ოპერაციული სისტემა ფაილს კითხულობს და:

- ამოწმებს, ახალი ფაილი ნამდვილია თუ გაფუჭებული (magic ველი)
- ხვდება, რომელი CPU-სთვის დაინერა (cpu_type)
- ხვდება, სად ჩაიტვირთოს მეხსიერებაში (mem_load)
- კითხულობს სექციებს (კოდი, მონაცემები)

1.2 MEF ჰედერი (mef_hdr)

ეს სტრუქტურა ფაილის სულ პირველ ბაიტებში ინახება:

```
typedef struct __attribute__((packed)) {  
    uint32_t magic;           // 0x66656D7F => "\x7fmef"  
    uint32_t mem_load;        // სად ჩაიტვირთოს მეხსიერებაში  
    uint32_t file_size;       // ფაილის სრული ზომა  
    uint8_t  cpu_type;         // CPU ტიპი (x86, x86-64, ARM...)  
    uint8_t  os_type;          // OS ტიპი (Linux, Windows, macOS...)  
    uint8_t  sect_count;       // სექციების რაოდენობა  
    uint8_t  code_index;       // კოდის სექციის ინდექსი  
} mef_hdr;
```

ველების ცხრილი

ველი	აღწერა
magic	"Magic number" - ფაილის ხელმოწერა. 0x7F + 'mef'. ამით OS ხვდება, MEF ფაილია.
mem_load	მეხსიერების მისამართი, სადაც ფაილი ჩაიტვირთება. ჩვენს შემთხვევაში: $4096 \cdot 40 = 163840$
file_size	ფაილის სრული ზომა ბაიტებში.
cpu_type	0x01 = x86, 0x02 = x86-64, 0x03 = AArch32, 0x04 = AArch64
os_type	0x01 = Linux, 0x02 = Windows, 0x03 = macOS
sect_count	სექციების რაოდენობა (ჩვენს ფაილში 2: კოდი + მონაცემები)
code_index	სექციების სიაში კოდის სექციის ინდექსი (ჩვენთვის 1)

1.3 სექციის ჰედერი (sec_hdr)

MEF ჰედერის შემდეგ მოდის სექციების ჰედერები. ყოველი სექცია აღწერს ფაილის ერთ ნაწილს:

```
typedef struct __attribute__((packed)) {
    uint32_t file_offset;    // სად იწყება სექცია ფაილში
    uint32_t file_size;      // სექციის ზომა ფაილში
    uint32_t memory_offset;  // მეხსიერებაში სად ჩაიტვირთება
    uint32_t memory_size;    // მეხსიერებაში ზომა
    uint32_t protect;        // წვდომის ფლაგები
} sec_hdr;
```

protect ფლაგები

ველი	აღნიშვნა
0x00000001	EXECUTABLE — სექცია კოდია, CPU-ს შეუძლია გაუშვას
0x00000010	WRITABLE — შეიძლება ჩანერა
0x00000100	READABLE — შეიძლება კითხვა

მაგ. კოდის სექცია ჩვეულებრივ: EXECUTABLE + READABLE = 0b00000101
მონაცემთა სექცია: WRITABLE + READABLE = 0b00000110

1.4 MEF ფაილის წაკითხვა C-ში (main.c)

main.c პროგრამა ასე კითხულობს MEF ფაილს:

```
// 1. გახსენი ფაილი
int fd = open("mini64.mef", O_RDONLY);

// 2. წაკითხე ფაილის ზომა
struct stat stat;
fstat(fd, &stat);

// 3. გამოყავი მეხსიერება
uint8_t *p_buff = malloc(stat.st_size);

// 4. წაკითხე ფაილი
read(fd, p_buff, stat.st_size);

// 5. ინტერპრეტირება: ბუფერის დასაწყისი = MEF ჰედერი
pmef_hdr p_hdr = (pmef_hdr)p_buff;

// 6. ჰედერის შემდეგ = სექციების ჰედერები
psec_hdr p_sec_hdr = (psec_hdr)(p_buff + sizeof(mef_hdr));
```

მნიშვნელოვანი ეს ტექნიკა ეწოდება "pointer casting" - ჩვენ ვეუბნებით C-ს: "ეს ბაიტები სინამდვილეში სტრუქტურაა". ასე მუშაობს ყველა ბინარული ფორმატის (ELF, PE, PNG, ZIP...) წაკითხველი.

2. Linux Kernel მოდული

Linux-ის ბირთვი (kernel) არის ოპერაციული სისტემის "გული". Kernel მოდული — ეს კოდია, რომელიც ბირთვს უმატებს ახალ შესაძლებლობებს პროგრამის გადატვირთვის გარეშე.

2.1 მოდული VS ჩვეულებრივი პროგრამა

ჩვეულებრივი პროგრამა	Kernel მოდული
main() ფუნქციით იწყება	module_init() ფუნქციით იტვირთება
user space-ში გაეშვება	kernel space-ში გაეშვება
printf()-ით ბეჭდავს	printk()-ით ბეჭდავს
error კოდებს ბრუნებს	0 = წარმატება, < 0 = შეცდომა

2.2 მოდულის სტრუქტურა (my_modute.c)

```
#include <linux/init.h>    // __init, __exit მაკროები
#include <linux/module.h>  // module_init(), module_exit()
#include <linux/kernel.h>  // printk()

// ინიციალიზაციის ფუნქცია – insmod-ზე გაეშვება
static int __init my_module_init(void) {
    printk(KERN_INFO "Hello world!\n");
    return 0;    // 0 = ჩატვირთვა წარმატებულია
}

// გათიშვის ფუნქცია – rmmod-ზე გაეშვება
static void __exit my_module_exit(void) {
    printk(KERN_INFO "Goodbye world!\n");
}

module_init(my_module_init);    // ვუთითებთ init ფუნქციას
module_exit(my_module_exit);    // ვუთითებთ exit ფუნქციას

MODULE_LICENSE("GPL");
```

2.3 საკვანძო კონცეფციები

__init და __exit მაკროები

__init ნიშნავს: "ეს ფუნქცია მხოლოდ ჩატვირთვისას გამოვიყენო, შემდეგ მესხიერება გავათავისუფლო".

__exit ნიშნავს: "ეს ფუნქცია მხოლოდ გათიშვისას გამოვიყენო".

printk() vs printf()

```
printk(KERN_INFO "შეტყობინება\n");
// ^-- KERN_INFO = "[ INFO ]" – ლოგის დონე
// სხვა დონეები: KERN_ERR, KERN_WARNING, KERN_DEBUG
```

სად ვხედავთ ?	dmesg tail ბოლო kernel შეტყობინებები ყურება sudo dmesg -w რეალურ დროში
---------------	--

2.4 მოდულის კომპილაცია და გამოყენება

```
# კომპილაცია (Makefile-ით)
```

```
make

# მოდულის ჩატვირთვა
sudo insmod my_modute.ko

# ლოგის ნახვა
dmesg | tail

# მოდულის გამოტვირთვა
sudo rmmod my_modute
```

3. x86-64 Assembly — mini64.asm

Assembly ენა — ყველაზე დაბალი დონის პროგრამირების ენაა. თითოეული ბრძანება პირდაპირ CPU-ს ინსტრუქციად გარდაიქმნება.

3.1 რეგისტრები (Registers)

x86-64 CPU-ს 16 ძირითადი 64-ბიტის რეგისტრი აქვს. ყველაზე ხშირად გამოყენებული:

ველი	აღწერა
rax	accumulator — შედეგები, syscall ნომერი
rdi	1-ლი არგუმენტი (destination index)
rsi	მე-2 არგუმენტი (source index)
rdx	მე-3 არგუმენტი (data)
rsp	stack pointer — სტეკის მაჩვენებელი
rip	instruction pointer — შემდეგი ბრძანება (ავტომატური)

3.2 System Call (syscall)

Linux-ში პროგრამა OS-ს სთხოვს სერვისებს syscall-ის გამოყენებით. mini64.asm-ში 2 syscall გამოიყენება:

write() — ეკრანზე ჩანერა

```
mov rax, 1      ; syscall ნომერი: 1 = write()
mov rdi, 1      ; fd=1 = stdout (ეკრანი)
mov rsi, msg_addr ; message-ის მისამართი
mov rdx, msg_size ; message-ის ზომა
syscall         ; OS-ს გამოძახება
```

exit() — გამოსვლა

```
mov rax, 60     ; syscall ნომერი: 60 = exit()
mov rdi, 0      ; exit code: 0 = წარმატება
syscall
```

3.3 MEF ფაილის ასამბლეა (mini64.asm)

mini64.asm ერთდროულად ქმნის MEF ჰედერს და პროგრამის კოდს. ფაილი 3 ნაწილად არის:

ნაწილი 1: MEF ჰედერი (ბაიტები ხელით)

```
db 0x7f, 'm', 'e', 'f' ; magic = 0x66656D7F
```

```

dd file_load_va      ; mem_load = 163840
dd file_end          ; file_size (ლეიბლი = მისამართი)
db 2                 ; cpu_type = x86-64
db 1                 ; os_type = Linux
db 2                 ; sect_count = 2 სექცია
db 1                 ; code_index = 1

```

ნაწილი 2: კოდის სექცია

```

times page_size - ($ - $$) db 0 ; padding პირველ გვერდამდე

code_start:
    mov rax, 1
    mov rdi, 1
    mov rsi, file_load_va + msg ; msg-ის ვირტუალური მისამართი
    mov rdx, msg_size
    syscall

    mov rax, 60
    mov rdi, 0
    syscall

```

ნაწილი 3: მონაცემთა სექცია

```

data_start:
    msg: db `Hello, world!\n`
    msg_size: equ $ - msg ; $ = მიმდინარე მისამართი
data_size: equ $ - data_start

```

equ vs db

db (define byte) — რეალური ბაიტი ფაილში ინახება. **equ** — მხოლოდ ასემბლერის "ლეიბლია", ფაილში ვერ ნახავ. მათემატიკური მნიშვნელობა კომპილაციისას.

3.4 MEF ფაილის გამართვა

```

# კომპილაცია
nasm -f bin -o mini64.mef mini64.asm

# ჰექს-ნახვა (ჰედერის შემოწმება)
xxd mini64.mef | head -5

# C პარსერის გაშვება
cd 02_mef_reader && make && ./reader

```

4. MEF Kernel Binary Format Handler (mef_format.c)

mef_format.c — ეს Linux kernel-ის მოდულია, რომელიც OS-ს ასწავლის MEF ფაილების პირდაპირ გაშვებას. ისევე, როგორც kernel იცის ELF ფაილის გაშვება, ჩვენი მოდულის შემდეგ kernel იცის MEF-ისაც.

კონცეფცია

linux_binfmt — Linux-ის interface, რომლითაც kernel-ს ახალი exe ფორმატი ემატება. ELF, script (#!), WASM — ყველა ამ interface-ით მუშაობს.

4.1 linux_binfmt სტრუქტურა

kernel-ს ვეუბნებთ: "ეს ფუნქცია გამოიძახე, როცა MEF ფაილი გასაშვებია":

```
static struct linux_binfmt mef_binfmt = {
    .module      = THIS_MODULE,
    .load_binary = mef_load_binary, // ჩვენი ფუნქცია
};

// მოდულის ჩატვირთვისას — kernel-ს ვარეგისტრირებთ
register_binfmt(&mef_binfmt);

// მოდულის გათიშვისას — გხსნით
unregister_binfmt(&mef_binfmt);
```

4.2 mef_load_binary() — ნაბიჯ-ნაბიჯ

kernel execve() syscall-ს დაძახებისას ეს ფუნქცია ეშვება. ვნახოთ ყოველი ეტაპი:

ნაბიჯი 1: Magic შემოწმება

```
p_mef_hdr p_hdr = (p_mef_hdr)bprm->buf;
// bprm->buf = ფაილის პირველი 128 ბაიტი (kernel-მა წაიკითხა)

if (p_hdr->magic != MEF_HDR_MAGIC) {
    return -ENOEXEC; // "ეს ფაილი ჩვენი არ არის"
}
```

ნაბიჯი 2: შედერების წაკითხვა kernel space-ში

```
// MEF შედერი stack-ზე
mef_hdr hdr_buf;
pos = 0;
kernel_read(bprm->file, &hdr_buf, sizeof(mef_hdr), &pos);

// სექციების შედერები heap-ზე (kvmalloc)
sec_hdr *sec_buf = kvmalloc(
    hdr_buf.sect_count * sizeof(sec_hdr),
    GFP_KERNEL
);
kernel_read(bprm->file, sec_buf,
    hdr_buf.sect_count * sizeof(sec_hdr), &pos);
```

რატომ
kvmalloc
?

kernel-ში malloc() არ არის. kvmalloc() — kernel-ის ეკვივალენტი. GFP_KERNEL = 'ჩვეულებრივი kernel alloc'. კვfree() = free().

ნაბიჯი 3: პროცესის ინიციალიზაცია

```
retval = begin_new_exec(bprm);  
// ეს ფუნქცია:  
// - ძველ address space-ს წმენდს  
// - ახალ virtual memory-ს ამზადებს  
// - signal handlers-ს აწყობს
```

ნაბიჯი 4: სექციების vm_mmap()

```
for (i = 0; i < hdr_buf.sect_count; i++) {  
    vm_mmap(bprm->file,  
            hdr_buf.mem_laod + sec_buf[i].memory_offset, // VA მისამართი  
            sec_buf[i].file_size,                        // ზომა  
            sec_buf[i].protect,                          // PROT_READ|EXEC  
            MAP_PRIVATE | MAP_FIXED,                    // fixed address  
            sec_buf[i].file_offset                       // ფაილში offset  
    );  
}  
// vm_mmap = user space-ის mmap()-ის kernel ვერსია
```

ნაბიჯი 5: Entry Point-ის დაყენება

```
unsigned long entry_point = hdr_buf.mem_laod +  
    sec_buf[hdr_buf.code_index].memory_offset;  
  
struct pt_regs *regs = current_pt_regs();  
start_thread(regs, entry_point, bprm->p);  
// ^ ეს RIP (instruction pointer) რეგისტრს entry_point-ზე მიმართავს  
// და user space-ში გადასტემა — პროგრამა იწყება!
```

4.3 mef_format.h — kernel + user space

განახლებულ mef_format.h-ში საინტერესო ტექნიკაა — ერთი header ორ კონტენტში მუშაობს:

```
#ifdef __KERNEL__  
    #include <linux/types.h> // kernel-ის uint8_t, uint32_t  
#else  
    #include <stdint.h>      // user space-ის uint8_t, uint32_t  
#endif
```

მნიშვნელ
ოვანი
ცვლილე
ბა

protect ფლაგები შეიცვალა Linux PROT_* კონსტანტებთან შესაბამისობისათვის: PROT_READ=0x1, PROT_WRITE=0x2, PROT_EXEC=0x4 ძველ ვერსიაში EXEC=0x1 იყო, ახალში EXEC=0x4. vm_mmap() პირდაპირ ამ მნიშვნელობებს იყენებს.

4.4 სრული სურათი — MEF execve() ნაკადი

ველი	აღწერა
1. execve("mini64.mef")	user space — პროგრამა გასაშვებია

2. kernel ეძებს handler -ს	kernel ამონმებს ყველა registered binfmt magic-ს
3. mef_load_binary() გამოიძახება	ჩვენი kernel მოდული პასუხობს
4. magic + file_size check	ვამოწმებთ, MEF ფაილია თუ შეცდომაა
5. begin_new_exec()	ძველი address space-ი იხოცება
6. vm_mmap() x სექციების რაოდ.	ყოველი სექცია მესხიერებაში ირუქება
7. start_thread(entry_point)	RIP = code სექციის დასაწყისი
8. პროგრამა გაეშვება!	mini64.asm-ის კოდი იწყებს შესრულებას

5. ყველაფერი ერთად — პროგრამის სიცოცხლის ციკლი

ახლა ვნახოთ, როგორ უკავშირდება ეს სამი თემა ერთმანეთს:

ნაბ.	ვინ მუშაობს	რა ხდება
1	NASM (Assembler)	mini64.asm კომპილდება → mini64.mef (ბინარი MEF ფორმატში)
2	C პროგრამა (main.c)	MEF ფაილს კითხულობს, ჰედერს ამონმებს, სექციებს ბეჭდავს
3	Linux Kernel	OS ქმნის სამუშაო გარემოს: მესხიერება, syscall, პროცესი
4	Assembly პროგრამა	syscall write() => ეკრანზე ბეჭდავს "Hello, world!"
5	Assembly პროგრამა	syscall exit() => პროგრამა სწორად სრულდება

5.1 პრაქტიკული სავარჯიშოები

- mini64.asm-ში შეცვალე "Hello, world!" შენი სახელით. გაუშვი ხელახლა.
- my_modute.c-ში printk-ში ჩაწერე შენი სახელი. ჩაყენე მოდული, dmesg-ში ნახე.
- main.c-ში დაამატე ახალი switch case — MEF_HDR_CPU_TYPE_RISCV (0x05).
- ნახე mini64.mef ჰექს-ედიტორში (xxd) და მოძებნე magic ბაიტები 7F 6D 65 66.
- mef_format.c-ში pr_info()-ით დაბეჭდე ჩატვირთული სექციების რაოდენობა.

6. ტერმინთა განმარტება

ველი	აღწერა
Binary / ბინარი	კომპიუტერის პირდაპირ გასაგები ფაილი (0 და 1)
Section / სექცია	ფაილის ლოგიკური ნაწილი: კოდი, მონაცემები, string-ები...
Virtual Address	პროგრამის საკუთარი მესხიერების მისამართი (OS ანაწილებს)
Syscall	პროგრამის მიმართვა OS-სთან (write, read, open, exit...)
Register	CPU-ს ულტრა-სწრაფი შენახვის ადგილი (rax, rdi...)

Kernel Space	OS-ის კოდის სამუშაო გარემო — პრივილეგირებული
User Space	ჩვეულებრივი პროგრამების სამუშაო გარემო — შეზღუდული
ELF	Linux-ის ოფიციალური exe ფორმატი (MEF-ის "ნამდვილი" ვერსია)
linux_binfmt	Kernel-ის interface ახალი exe ფორმატის დასარეგისტრირებლად
linux_binprm	Kernel-ის სტრუქტურა, execve() -ის დროს ფაილის ინფო
bprm->buf	ფაილის პირველი 128 ბაიტი — magic შემოწმებისთვის
vm_mmap()	kernel-ის ფუნქცია: ფაილის სექცია virtual memory -ზე არარუქება
kvmalloc/kvfree	kernel-ის malloc/free — GFP_KERNEL ფლაგით
pt_regs	CPU რეგისტრების სტრუქტურა kernel -ში
start_thread()	RIP-ს ადგენს entry point -ზე — პროგრამა იწყება
insmod / rmmmod	Kernel მოდულის ჩასმა / გამოსმა
dmesg	Kernel-ის ლოგის ნახვის ბრძანება

წარმატება სწავლაში!